南通大学人工智能与计算机学院

课程名称：汇 编 语 言 程 序 设 计

实验名称：实验 1-2 汇编语言基本指令

任课教师：鲁　松

实验日期：2026 年 10 月 22 日

学生姓名：林志豪 班级：计 252 学号：2542110155

一、目的及要求

本次实验编号为 1-2，选做第 2、3、4、5、7 题。练习字变量交换、双字高低 16 位读取、字节数组寻址、按位与运算，以及 CX 寄存器高低字节交换。

熟悉 8086 汇编程序的数据段、堆栈段和代码段结构，继续练习使用 MASM 编译、LINK 链接和 DEBUG 单步执行，并将实际运行结果与预期值进行比较。

实验报告列出每题的完整程序文字，使用实际屏幕截图记录运行结果，并按要求导出为 PDF。

二、环境（软、硬件平台）

硬件平台：能够运行 Windows 和 DOSBox 的个人计算机。

软件平台：Windows 操作系统、DOSBox 0.74、Microsoft Macro Assembler 5.00、Microsoft Overlay Linker 3.60、DEBUG 及记事本。

工作目录：D:\Study\My_Project\Assembly-language\lab1_2。在记事本中编辑并保存.asm 源文件，将该目录挂载为 DOSBox 中的 C 盘。

三、内容及步骤（包括程序流程及说明）

通用流程：编辑源程序 → 编译生成.obj 目标文件 → 链接生成.exe 可执行文件 → 用 DEBUG 调试.exe 文件 → 观察寄存器和内存 → 截图记录结果。

DOSBox 工作目录挂载命令如下：

```
mount c "D:\Study\My_Project\Assembly-language\lab1_2"
c:
```

各题先设置 DS，再执行核心指令。DEBUG 中，t 用于单步执行，r 用于查看寄存器，d 用于查看内存，q 用于退出调试。查看内存时按字节读取，并按小端存储顺序解释字和双字数据。

第 3 题要求高 16 位进入 AX、低 16 位进入 DX。所有结果均在核心指令执行后记录；退出前设置 AX=4C00h 的指令暂不执行，以免影响对 AX 的观察。

三、内容及步骤（续）

第 2 题　交换字变量 x 和 y

题目及测试数据：数据段有字变量 x 和 y，交换它们的值。初值设置为 x=1234h、y=5678h。

思路：分别将 x 和 y 读入 AX、BX，再交叉写回变量，避免覆盖变量后丢失原值。

程序流程：初始化 DS → 读取 x 到 AX → 读取 y 到 BX → BX 写入 x、AX 写入 y → 退出。

完整程序　ex2.asm

```
data segment
    x dw 1234h
    y dw 5678h
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    mov ax, x
    mov bx, y
    mov x, bx
    mov y, ax

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex2.asm; → link ex2.obj; → debug ex2.exe

调试步骤：先执行 t 2，用 d ds:0 1 4 观察初值；再执行 t 4，用同一命令查看交换后的数据。

预期：x=5678h、y=1234h；内存由 34 12 78 56 变为 78 56 34 12。

三、内容及步骤（续）

第 3 题　读取双字的高低 16 位

题目及测试数据：数据段定义双字变量 value32，将高 16 位送入 AX，低 16 位送入 DX。测试初值为 12345678h。

思路：双字按小端方式存储。value32 处的字为低 16 位 5678h，value32+2 处的字为高 16 位 1234h；使用 WORD PTR 明确按字读取。

程序流程：初始化 DS → value32+2 处的字送 AX → value32 处的字送 DX → 退出。

完整程序　ex3.asm

```
data segment
    value32 dd 12345678h
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    mov ax, word ptr value32+2
    mov dx, word ptr value32

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex3.asm;　→　link ex3.obj;　→　debug ex3.exe

调试步骤：执行 t 4，再用 r 查看寄存器，用 d ds:0 1 4 查看双字的 4 个字节。

预期：AX=1234h、DX=5678h；内存中的 4 个字节为 78 56 34 12。

三、内容及步骤（续）

第 4 题　复制数组第 4 个元素

题目及测试数据：字节数组 scores 有 6 个元素，将第 4 个元素（下标 3）复制到 temp。数组初值依次为 10、20、30、40、50、60，temp 初值为 0。

思路：每个数组元素占 1 字节，下标 3 对应 scores+3。先读入 AL，再由 AL 写入 temp。十进制 40 对应十六进制 28h。

程序流程：初始化 DS → scores+3 处的字节送 AL → AL 写入 temp → 退出。

完整程序　ex4.asm

```
data segment
    scores db 10, 20, 30, 40, 50, 60
    temp db 0
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    mov al, scores+3
    mov temp, al

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex4.asm; → link ex4.obj; → debug ex4.exe

调试步骤：先执行 t 2，用 d ds:0 1 7 观察数组和 temp；再执行 t 2 并查看同样的 7 个字节。

预期：数组不变，temp 从 0 变为 40（28h），即最后一个字节从 00 变为 28。

三、内容及步骤（续）

第 5 题　保留字变量最低 4 位

题目及测试数据：将字变量 flags 的最低 4 位保留，其余位全部清零。程序中用 flagsValue 表示该变量，测试初值为 FFFFh。

思路：000Fh 的二进制为 0000 0000 0000 1111。与其进行 AND 运算后，原数最低 4 位保持不变，其余 12 位清零。

程序流程：初始化 DS → flagsValue 与 000Fh 按位与 → 结果保存到原变量 → 退出。

完整程序　ex5.asm

```
data segment
    flagsValue dw 0FFFFh
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    and flagsValue, 000Fh

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex5.asm; → link ex5.obj; → debug ex5.exe

调试步骤：先执行 t 2，用 d ds:0 1 2 查看原值；再执行一次 t，用同一命令查看结果。

预期：flagsValue 从 FFFFh 变为 000Fh；内存中的 FF FF 变为 0F 00。

三、内容及步骤（续）

第 7 题　交换 CX 的高低字节

题目及测试数据：交换 CX 寄存器的高 8 位 CH 和低 8 位 CL。将 CX 初值设置为 1234h，即 CH=12h、CL=34h。

思路：使用 XCHG CH,CL 交换两个 8 位寄存器。操作后 CH=34h、CL=12h，因此 CX=3412h。

程序流程：初始化 DS → 1234h 装入 CX → 交换 CH 与 CL → 退出。

完整程序　ex7.asm

```
data segment
    initialValue dw 1234h
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    mov cx, initialValue
    xchg ch, cl

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex7.asm; → link ex7.obj; → debug ex7.exe

调试步骤：执行 t 3，观察 CX=1234h；再执行一次 t，观察 CX 是否变为 3412h。

预期：CX 从 1234h 变为 3412h。

四、运行结果

第 2 题　交换字变量 x 和 y

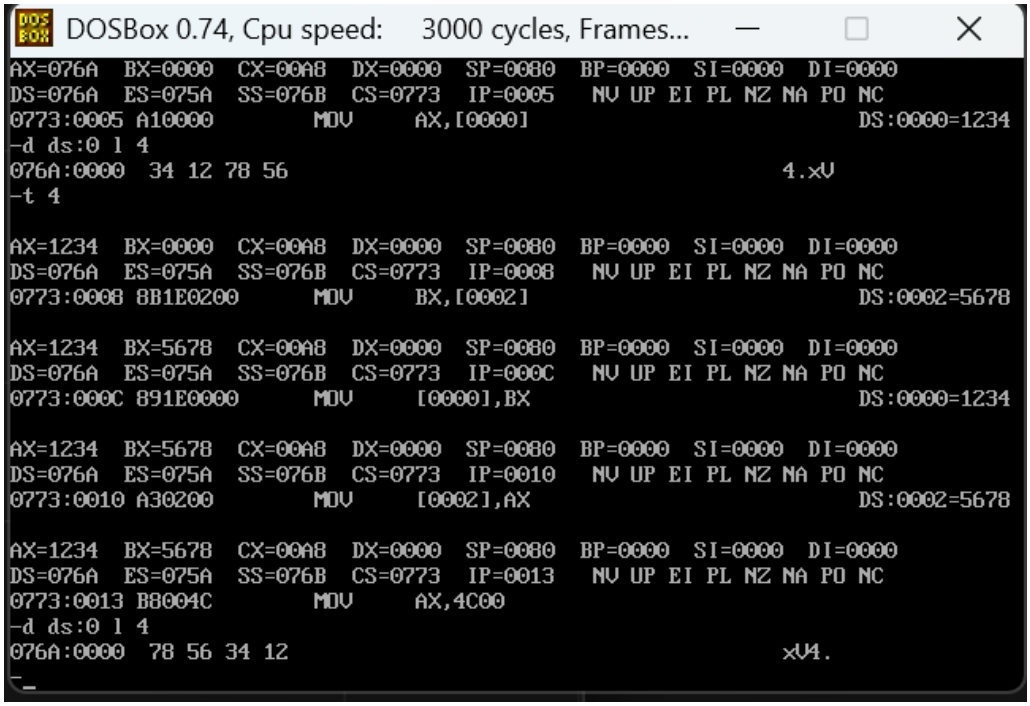


图 1　第 2 题交换前后的内存数据

结果：34 12 78 56 变为 78 56 34 12，即 x=5678h、y=1234h，两个字变量交换成功。

第 3 题　读取双字的高低 16 位

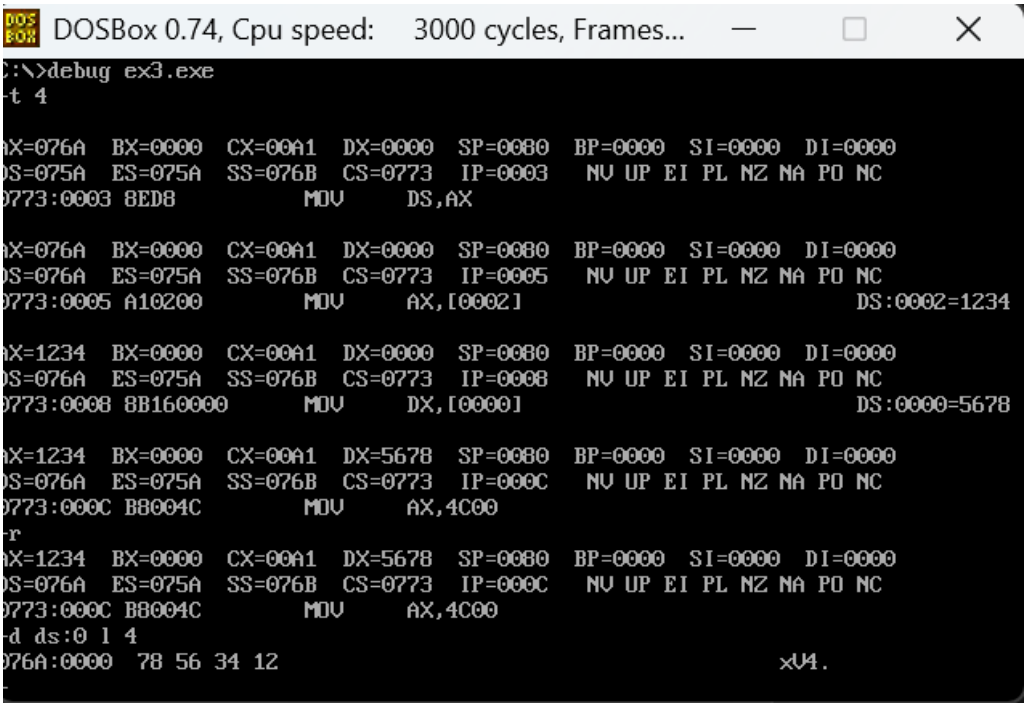


图 2　第 3 题 AX、DX 及 value32 的内存数据

结果：AX=1234h、DX=5678h，内存为 78 56 34 12，高 16 位和低 16 位均读取正确。

四、运行结果（续）

第 4 题　复制数组第 4 个元素

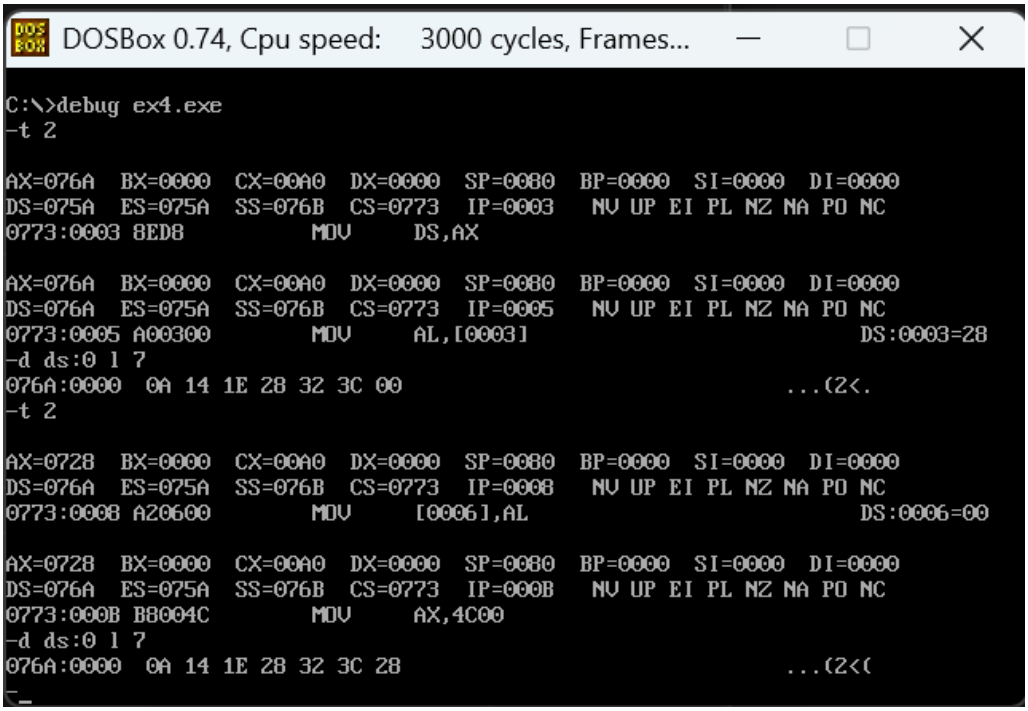


图 3　第 4 题复制前后的数组及 temp 数据

结果：数组前 6 个字节不变，最后的 temp 从 00 变为 28，与第 4 个元素相同，复制成功。

第 5 题　保留字变量最低 4 位



图 4　第 5 题按位与运算前后的数据

结果：FF FF 变为 0F 00，即 FFFFh 变为 000Fh，最低 4 位保留，其余位清零。

四、运行结果（续）

第 7 题　交换 CX 的高低字节

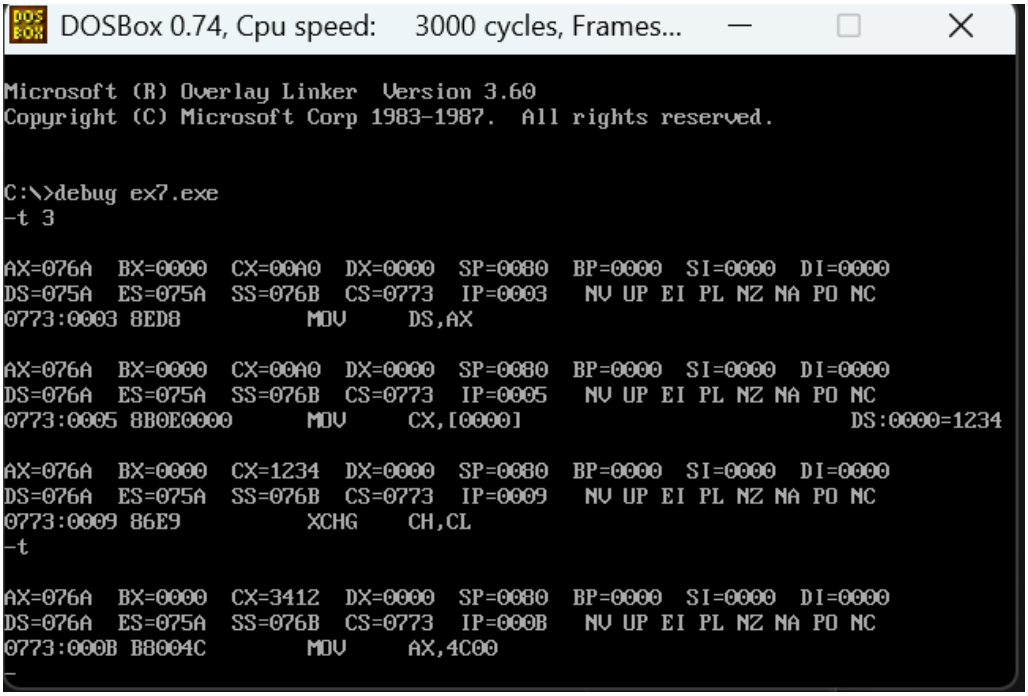


图 5　第 7 题交换前后的 CX 寄存器值

结果：CX 从 1234h 变为 3412h，CH 和 CL 交换成功。五题的实际运行结果均与预期一致。

五、问题及心得

通过本次实验，我进一步熟悉了编辑、编译、链接和单步调试的过程。调试时根据程序中的指令条数控制单步执行位置，避免将下一条尚未执行的指令误认为已经执行。

本次还加深了对字节数组下标、小端存储和按位与运算的理解。双字的高低 16 位要根据题目要求送入指定寄存器；数组第 4 个元素对应下标 3；保留最低 4 位应使用 000Fh 作为掩码。